

業務マニュアル 管理システム

設計概要

業務マニュアルを共有・管理する
Webアプリケーション



属人化しがちな業務ナレッジを
一元管理し、正確な情報共有を実現します。



本システムにより、作成・承認・公開・履歴管理を
一元的に行い、業務の透明性と効率を向上させます。



情報の属人化

- ・担当者しか知らない業務が出来てしまう
- ・業務負荷が担当者に集中してしまう。



履歴が追えない

- ・変更内容や経緯が残らず、確認が困難
- ・そのルールになった理由が不明のままになる



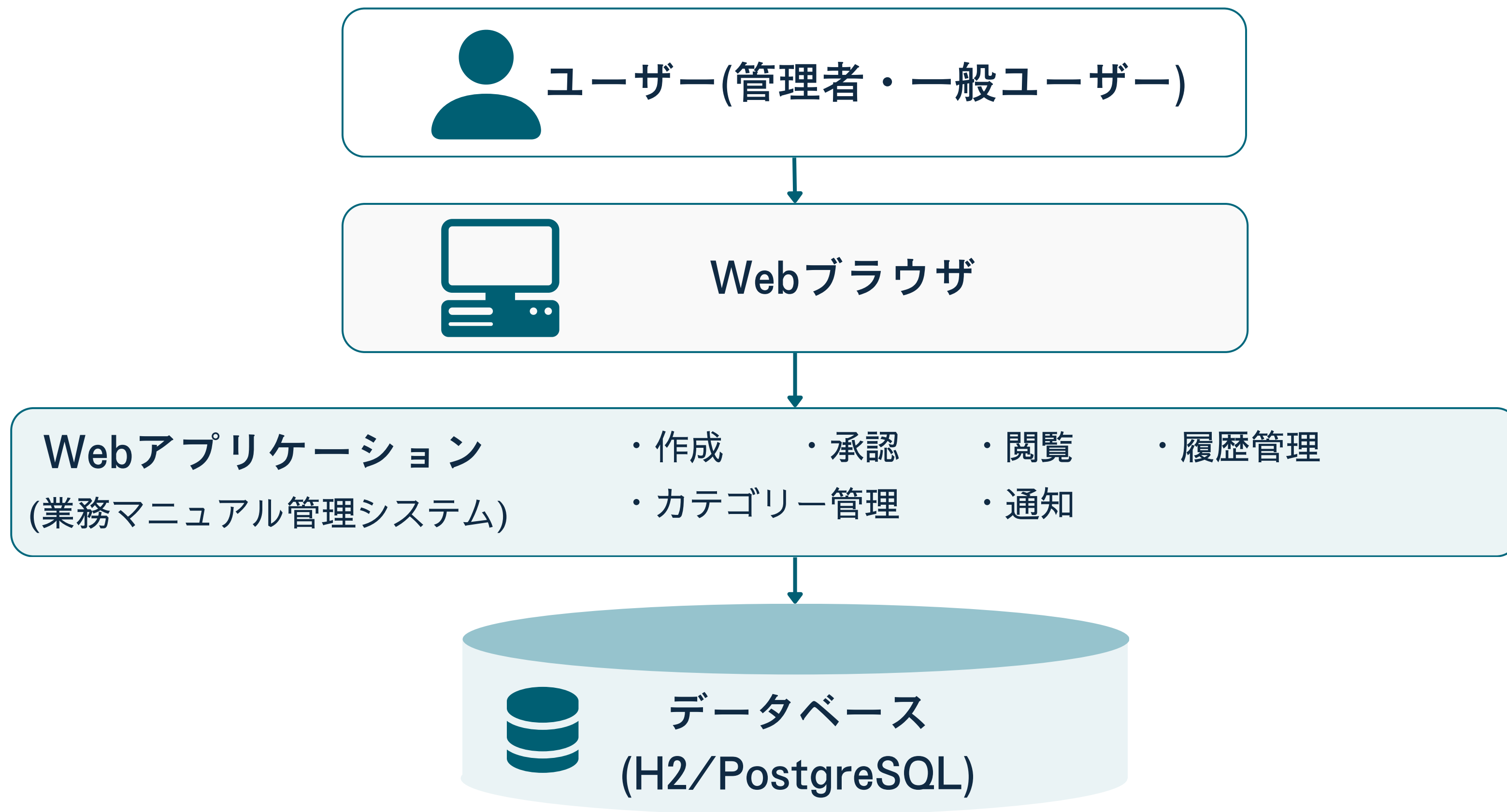
最新情報が分からない

- ・変更状況が不明で、
古い情報を参照してしまう。



承認状態が不明瞭

- ・誰が承認したか、現在の状態が分からない
- ・部署で認められたルールか個人のルールか分からない



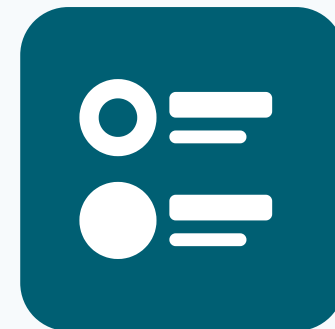
Backend



Java 17 / Spring Boot 4.0.5

- Spring MVC
- Spring Data JPA
- Spring Security

Frontend



Thymeleaf / Bootstrap 5

- レスポンシブ対応
- UIコンポーネント活用

Database



H2(開発環境) /
PostgreSQL(本番環境)

- JPA/Hibernate

その他

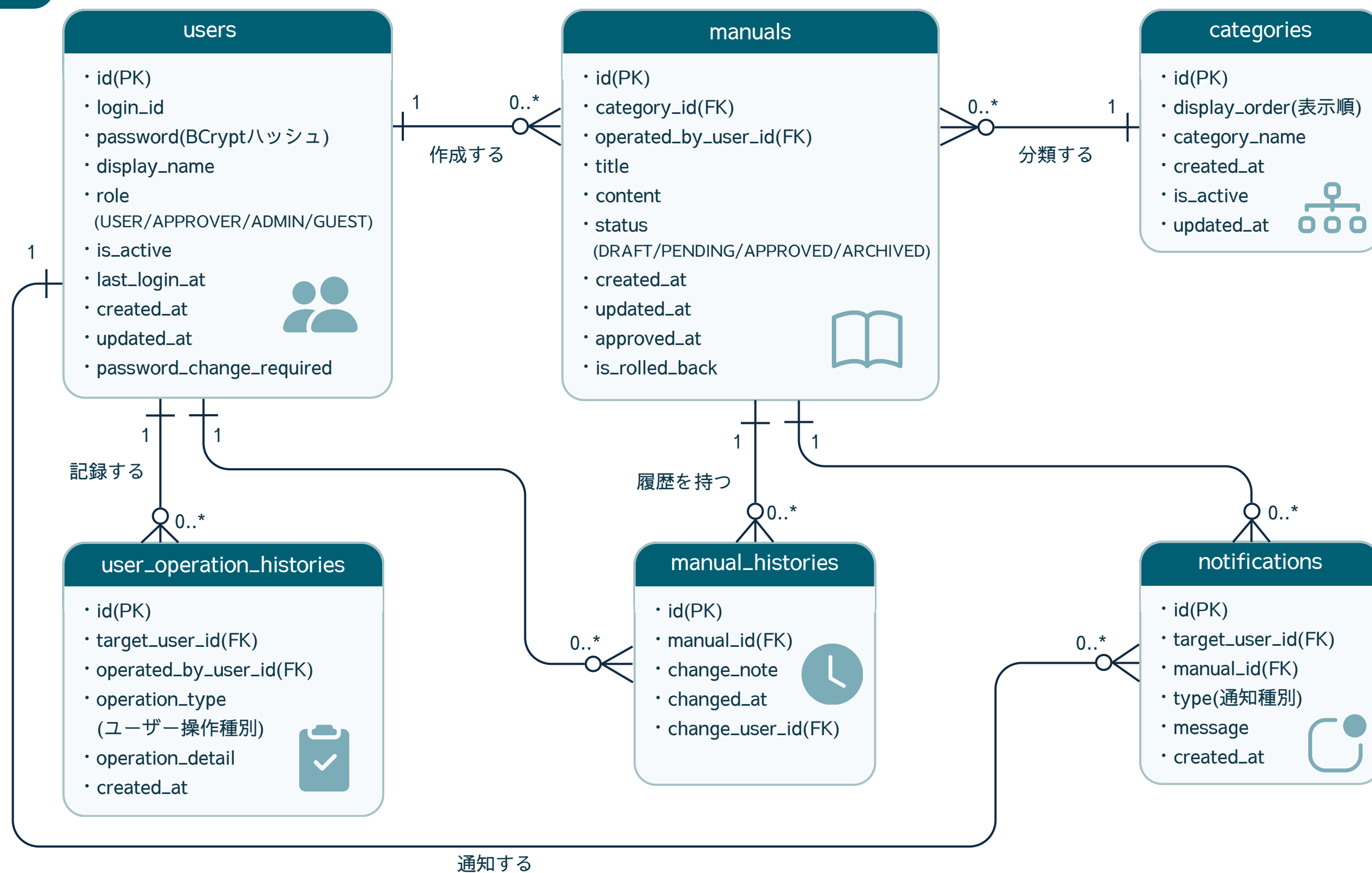


Maven/Git/GitHub

- Render(Web公開・DB運用)
- DBeaver(DB接続/確認)
- Postman(API/HTTP動作確認)

ER図(データベース設計)

主要なエンティティとリレーションを示します。



設計のポイント

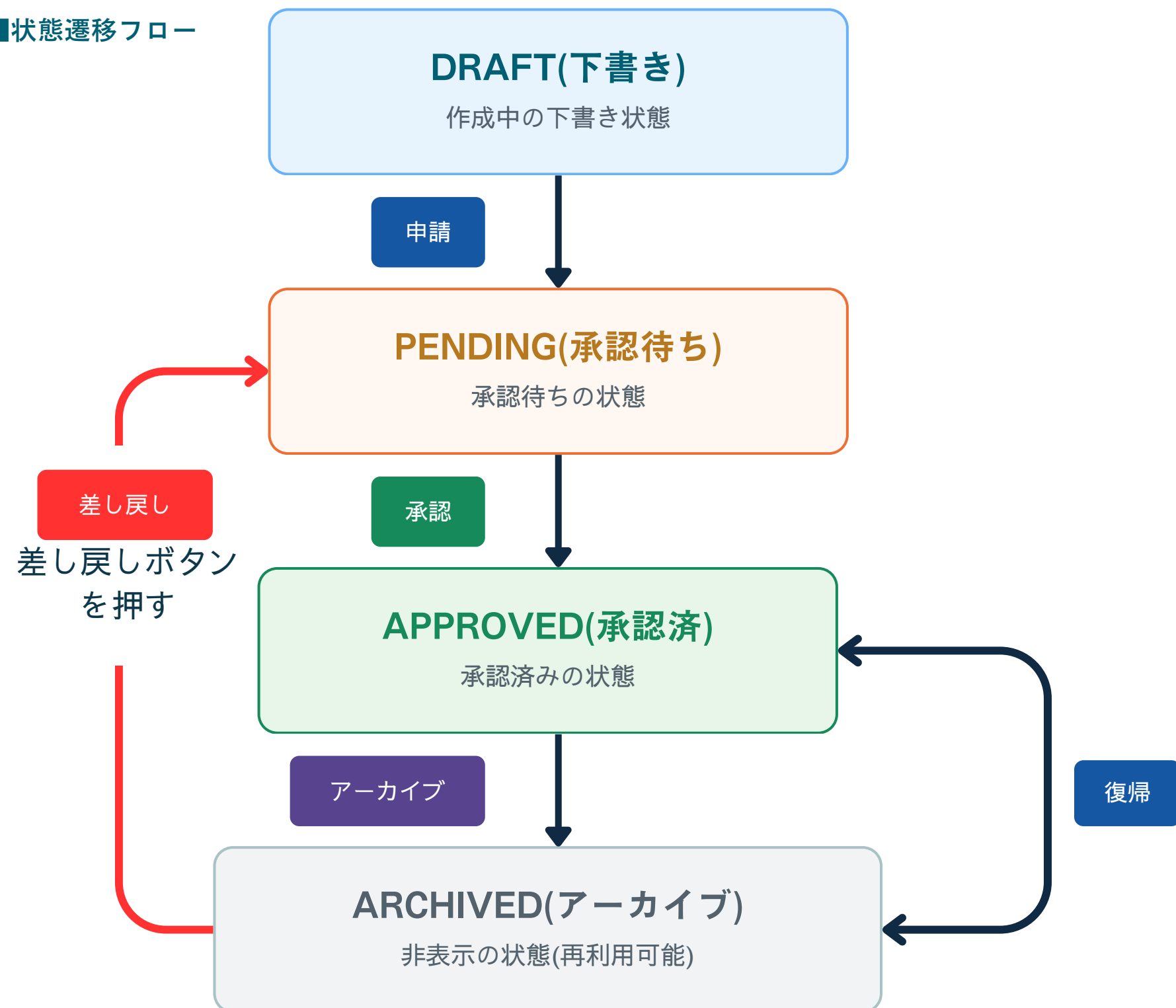
- Manual本体とManualHistoryを分離 (履歴はappend-only)
- Manualはstatusで現在状態を管理
- 通知はUserとManualに紐づく
- 管理操作はUserOperationHistoryに記録
- FK制約で孤児データを防止(manuals/categories/users/histories/notificationの整合性維持)
- ManualHistoriesとUserOperationHistoriesを分離し、業務変更履歴と管理操作履歴を明確化
- manuals.statusを列挙値で管理し、状態遷移ルールをService層で制御
- users/categoriesはis_activeで無効化し履歴参照性を維持
- notificationsは既読フラグを持たず、削除運用でシンプル性を優先

■表記の説明

- PK : Primary Key (主キー)
- FK : Foreign Key (外部キー)

1 + — o — 0..* : 1対多の関係

■状態遷移フロー



■ルール・注意点

- ・作成者本人は承認不可
- ・承認時にapproved_atを保存
- ・差し戻し時にis_rolled_back=true
- ・再申請時にis_rolled_back=false
- ・物理削除は行わず、ARCHIVEDで保持

■状態の定義

- ・DRAFT:
下書き/承認前
- ・PENDING:
承認待ち/承認者確認中
- ・APPROVED:
正式承認済み/通常利用対象
- ・ARCHIVED:
アーカイブ/旧版(履歴・過去参照用に保持)



通知との連動

申請・差し戻し時に
関係者へ通知を送信します。



履歴との連携

状態変更のたびに
manual_historiesに記録します。



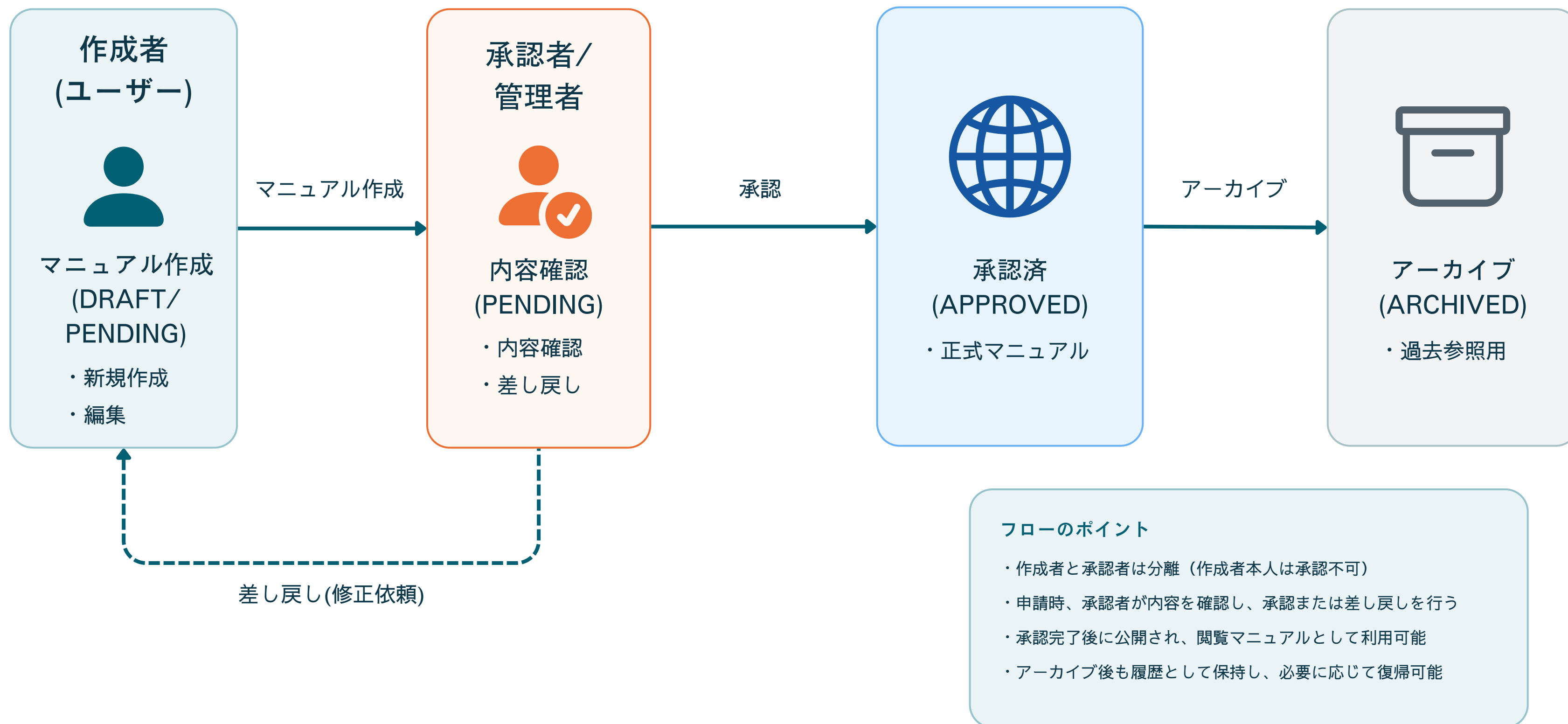
再利用のしやすさ

ARCHIVEDから復帰でき、
過去資産を活用できます。



データ保持方針

削除は行わず、
ARCHIVEDで保持し
参照性を維持します。





認証：Spring Security(フォームログイン)

Spring Securityによる認証を実装。
パスワードはハッシュ化(BCrypt)して安全に管理しています。



認可：ロールベース制御(RBAC)

USER/APPROVER/ADMIN/GUESTの4ロールを定義し、
ユーザーごとに操作可能な機能を制御しています。



権限制御：二重チェック(UI+Service層)

画面上のボタン・リンクの制御に加え、Service層でも権限と
業務ルールをチェックし、不正な操作を防止します。



入力検証：バリデーション

@Validによる入力チェックを行い、不正な値や不正なデータの
登録を防止します。



業務ルールによる不正操作防止

ステータス遷移の制御や「作成者本人は承認不可」などの
業務ルールを実装し、意図しない操作を防止します。



CSRF対策

Spring SecurityのCSRF機能を有効化し、クロスサイトリクエスト
フォージェリ攻撃を防止します。



データ保護・整合性

トランザクション管理により整合性を担保し、重要データの
不整合や改ざんを防止します。

アクセス制御のイメージ



想定される脅威と対策	
想定される脅威・不正操作	対策
他人のマニュアルを不正に承認する	ロールチェック+作成者チェックにより承認を制限
URLを直接入力して不正な画面へアクセスする	Service層で権限・ステータスを再チェックし、不正操作を拒否
不正なデータを送信する(改ざん・不正入力)	@Validによる入力検証で不正データを防止
CSRF攻撃により意図しない操作を行う	Spring SecurityのCSRF対策で攻撃を防止
データの改ざん・不整合が発生する	トランザクション管理と業務ルールでデータの整合性を担保



ステータス管理

DRAFT/PENDING/APPROVED/ARCHIVEDのステータスで、マニュアルのライフサイクルを明確化。承認状況が一目で分かり、誤公開を防止します。



履歴分離・透明性の確保

Manual本体とManualHistoryを分離し、変更履歴をappend-onlyで管理。いつ・誰が・何を変更したかを追跡でき、業務の透明性と説明責任を担保します。



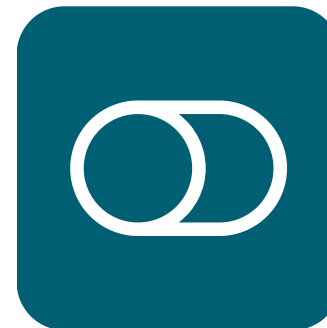
権限分離と承認制御

作成者と承認者を分離し、作成者本人は承認不可。承認フローと権限分離により、内部統制と品質の担保を実現します。



新規タブ表示の工夫

マニュアルを新規タブで表示することで、業務画面を閉じずに参照可能に。複数マニュアルの比較や一覧を開いたまま閲覧ができ、既存業務を妨げません。



更新履歴の活用

運用ルールの変遷や意思決定の経緯を確認可能。ルール整備や改善の参考となり、ナレッジの蓄積と業務改善に役立ちます。



同名チェック（モーダル確認）

同名カテゴリやタイトル登録時にモーダルで確認。誤登録を防止しつつ、必要に応じて同名保存や旧版アーカイブなど柔軟に対応できます。



モード切替（新規/変更モード）

新規作成モードと変更モードを切り替えることで、画面をシンプルに。連続登録や編集をスムーズに行える設計で、操作効率を向上させています。

📌 今後の拡張予定

お気に入りマニュアル機能

良く使うマニュアルをすぐに参照可能に

ユーザー検索機能

キーワードや属性でユーザーを素早く検索

ユーザーとカテゴリの紐づけ

部署・役割に応じた閲覧・管理権限を強化

正式マニュアル作成支援機能

テンプレートや入力補助で作成を効率化

**ナレッジの一元管理**

業務マニュアルを一元管理し、最新版を全社で共有。
属人化を防ぎ、業務品質を標準化します。

**透明性と説明責任の確保**

履歴管理により変更の経緯や責任の所在を明確化。
監査対応やトラブル時の迅速な原因特定にも貢献します。

**品質とコンプライアンスの向上**

承認フローと権限管理により、誤公開や不適切な情報発信を防止。
内部統制とコンプライアンスを強化します。

**スケーラブルな運用基盤**

拡張性の高い設計により、今後の機能追加や組織拡大にも
柔軟に対応可能な運用基盤を提供します。

**業務効率の向上**

検索性・一覧性・新規タブ閲覧により、
必要な情報へ素早くアクセス可能。
業務のスピードと生産性を向上します。

**期待する効果**

情報の属人化を解消し、組織全体の生産性を向上
正確で信頼性の高い情報を継続的に提供
安全で効率的な業務運用を支える基盤を構築